PRODUCT RESEARCH REPORT

OAuth & Authentication Architecture in Lokus

Product Research, Root Cause Analysis, and Remediation Specification for Issue #252

ISSUE REFERENCE	TARGET PLATFORMS	PRODUCT VERSION	DOCUMENT TYPE
GitHub #252	Windows, macOS, Linux	v1.3.0+ (Tauri 2.0)	Technical Research

SECTION 1

Executive Overview & Problem Statement

Lokus is an open-source, local-first markdown knowledge base and workspace platform powered by Tauri and React. To maintain user sovereignty over notes while enabling multi-device sync and productivity workflows, Lokus integrates a dual-tier identity model: local Guest Mode by default, paired with cloud-enabled Supabase GoTrue authentication and third-party integrations (Google Calendar, Gmail, Microsoft Graph, and Apple OAuth).

In GitHub Issue #252, end users attempting to authenticate via the "Continue with Google" button reported a hard blocker halting the onboarding experience with the following error payload:

REPORTED ERROR PAYLOAD — ISSUE #252

{"code": 400, "error_code": "validation_failed", "msg": "Unsupported provider: provider is not enabled"}

This research describes the underlying mechanics of this defect, explores the user experience impact, analyzes the root cause taxonomy across Supabase GoTrue and native Tauri desktop bindings, and documents the engineering blueprint deployed to resolve Google OAuth across all supported desktop platforms.

SECTION 2

User Personas, Journey Mapping & UX Friction

Authentication reliability is fundamental to Lokus onboarding and cloud feature adoption. Three primary product personas rely on this flow:

- **Knowledge Worker & Researcher:** Requires seamless 1-click Google authentication to store research notes, sync across across work and personal devices, and link Google Calendar for daily planning.
- **Privacy-First Note Taker:** Operates primarily in local Guest Mode but selectively connects Google Calendar/Gmail integrations through scoped OAuth permissions without compromising local data.
- **Self-Host & Open Source Contributor:** Runs local development instances with mock Supabase endpoints or Dockerized Dockerized Supabase CLI; expects authentication flows to operate predictably without crashing.

User Experience Gaps in Defective State

When Google OAuth failed with "validation_failed: provider is not enabled", users encountered total onboarding failure. The raw JSON string leaked internal backend mechanics to non-technical users, provided no clear recovery steps, and failed to guide users to email sign-in or guest mode.

SECTION 3

Technical Root Cause Analysis & Architectural Gaps

Detailed analysis of the authentication subsystem identified a multi-layered chain of gaps across backend configuration, CI deployment, runtime bindings, and client error handling:

- **Gap 1 — Missing GoTrue External Provider Declaration:** In Supabase Auth (GoTrue), third-party identity providers must must be explicitly registered under [auth.external.<provider>] in supabase/config.toml. While [auth.external.apple] was scaffolded, [auth.external.google] was absent. When the frontend called signInWithOAuth({ provider: "google" }), GoTrue rejected the request with HTTP 400 validation_failed.
- **Gap 2 — Omission of Desktop Redirect URI Allowlists:** Desktop OAuth flows rely on local loopback HTTP listeners listeners (http://127.0.0.1:9080/auth-callback) and custom deep links (lokus://auth-callback). The Supabase configuration only allowed https://127.0.0.1:3000, creating a secondary failure even if Google were enabled.
- **Gap 3 — Disconnect Between Build-Time and Runtime Environment Variables:** In native desktop binaries, std::env::v std::env::var("GOOGLE_CLIENT_ID") performs a runtime process lookup on the end-user machine. End users do not possess developer secrets in their OS environment. CI workflows passed secrets during compilation, but because the Rust backend lacked option_env! compile-time embedding, the client ID was missing at runtime.
- **Gap 4 — Multi-Platform CI Workflow Secret Ingestion:** While release.yml forwarded secrets for tagged releases, .github/workflows/build-multi-platform.yml built binaries without passing GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET, or Supabase credentials to tauri build.
- **Gap 5 — Local Development Network Traps:** In developer mode (VITE_LOCAL_DEV_MODE or dummy Supabase endpoints), initiating OAuth attempted real HTTP handshakes with non-existent dummy endpoints rather than simulating local developer credentials.

SECTION 4

Comparative Architecture State Matrix

Dimension	Pre-Fix State (Broken #252)	Post-Fix State (Engineered)
Supabase Config	Google provider missing from config.toml	[auth.external.google] enabled with skip_nonce_check
Redirect URIs	Only https://127.0.0.1:3000 permitted	Includes 127.0.0.1:9080, localhost, and lokus://
Client ID Lookup	Runtime-only std::env::var (missing in user OS)	option_env! compile-time baking + runtime override
Cargo Build Hook	No rerun-if-env-changed tracking	Cargo detects env updates and rebuilds crate
CI Pipeline	build-multi-platform.yml omitted secrets	Full secret propagation across all build runners
Frontend UX	Raw GoTrue JSON error shown in UI	User-friendly error translation with email fallback
Local Dev Flow	Crashes on dummy URL HTTP requests	Automatic local developer session bypass

SECTION 5

Technical Remediation & Implementation Blueprint

The remediation was implemented across 5 coordinated layers to guarantee resilience across cloud, native desktop, and local development environments:

1. Supabase Declarative Configuration Layer (supabase/config.toml)

Registered the Google external provider with skip_nonce_check enabled (required for native browser loopback authentication) and allowlisted all native redirect URIs:

- **Provider Definition:** [auth.external.google] enabled with client_id = "env(SUPABASE_AUTH_EXTERNAL_GOOGLE_CLIENT_ID)" and secret substitution.
- **Allowlisted Redirects:** Added http://127.0.0.1:9080/auth-callback, http://localhost:9080/auth-callback, and lokus://auth-callback.

2. Native Rust Runtime Binding (src-tauri)

Implemented dual-resolution credential retrieval across all Google and Microsoft connectors, combining runtime process lookup with compile-time binary embedding:

- **Dual-Resolution Pattern:** std::env::var("GOOGLE_CLIENT_ID").ok().or_else(|| option_env!("GOOGLE_CLIENT_ID").map(String::from)).filter(|s| !s.is_empty()).ok_or_else(...)
- **Cargo Cache Invalidation:** Emitted cargo:rerun-if-env-changed directives in src-tauri/build.rs to ensure binary freshness.

3. Frontend UX & Error Translation (src/core/auth/AuthManager.js)

Enhanced signInWithGoogle and signInWithApple to trap validation_failed and provider is not enabled HTTP 400 responses, converting them into intelligible guidance recommending email sign-in or admin configuration, while providing zero-overhead local development mocking.

VERIFIED RESILIENCE ARCHITECTURE

Desktop users running packaged releases automatically use the baked client ID. Enterprise users or self-hosters can override credentials at runtime via OS environment variables without modifying binary code.

SECTION 6

Quality Assurance & Automated Verification

The remediation underwent multi-tier validation across automated test suites, bundling verification, and runtime regression checks:

- **Unit Testing:** Implemented src/core/auth/AuthManager.test.js with mock Supabase responses to verify provider disabled disabled exception handling, Apple auth translation, and OAuth option parameters.
- **Frontend Bundle Integrity:** Verified full production build via Vite (npm run build) transforming 7,650+ modules without without bundling errors.
- **Platform Matrix Coverage:** Updated .github/workflows/build-multi-platform.yml to pass Google and Supabase secrets secrets across Windows, macOS, and Linux runners.

SECTION 7

Strategic Product Recommendations & Roadmap

To maintain high authentication reliability and safeguard against similar regressions in future releases, the following architectural practices are recommended:

- **Standardize Public Client IDs for PKCE:** For public desktop OAuth applications using PKCE, bake client IDs into official official production builds as a first-class constant, reserving environment variables strictly for self-hosted instances.
- **Pre-flight Auth Provider Diagnostics:** Introduce a lightweight background ping to the auth service upon app boot, dynamically disabling inactive OAuth buttons in LoginScreen.jsx with an informative tooltip before the user encounters an error.
- **Unified Token Lifecycle Management:**Ensure all calendar and email background connectors utilize the unified AuthManager token refresh lock to eliminate concurrent token refresh race conditions.
- **Telemetry & Anomaly Detection:** Log sanitized authentication failures (redacting tokens, query parameters, and PII) to PII) to GlitchTip/Sentry to capture provider outages or configuration mismatches before user reports.

Security & Compliance Checklist

Security Dimension	Protocol Standard	Implementation Status
OAuth Flow	OAuth 2.0 with PKCE (RFC 7636)	Enforced (SHA-256 code challenge)
Credential Storage	OS Keyring (Keychain / DPAPI / Secret Service)	Encrypted via SecureStorage in Rust
Loopback Origin	127.0.0.1 (Strict single-use local port)	Active on port 9080 with auto-close
Session Persistence	Encrypted local store with refresh rotation	Configured with 10s grace interval
Secret Hygiene	Zero secrets committed to version control	Environment variable substitution only

DOCUMENT APPROVAL & CONTROL

Status: Verified & Approved for Release | PR Branch: fix-252-broken-oauth-google | Reference Commit: d1b13b1 | Author: Clean Agent, Lokus Core Engineering